B.Tech CSE - Third Year | Lab 6

Assignment 6: Binary Search on a Sorted Array • *Aim: Search a sorted array by repeatedly halving the search range.*

Experiment No. 6.1: Binary search for a present key

1. **Experiment Title:** Binary search for a present key
2. **Aim:** Implement the task specified in the BCS2024545 lab assignment.
3. **Software / Tools Required:** Java JDK, text editor/IDE (IntelliJ IDEA, Eclipse, VS Code), terminal/command prompt.
4. **Theory:** Binary search works on sorted data and halves the candidate range after each comparison.
5. **Algorithm / Procedure:** Set low and high bounds; inspect the middle element; discard the half that cannot contain the key; repeat until found or bounds cross. Compile and execute.
6. **Program:**

```
public class Task_01_Binary_search_for_a_present_key {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 40));
    }
}
```

7. **Input:** Fixed input values declared in program: sorted array {10, 20, 30, 40, 50} and search key 40.
8. **Output & Complexity:**

```
3
```

   Time Complexity: O(log n) time | Auxiliary Space: O(1).
9. **Result:** Program implemented successfully and produced expected output.
10. **Viva-Voce Questions:** What condition must the array satisfy? How is mid calculated safely? Why does binary search discard half the range?

Experiment No. 6.2: Binary search for an absent key

1. **Experiment Title:** Binary search for an absent key
2. **Aim:** Implement the task specified in the BCS2024545 lab assignment.
3. **Program:**

```
public class Task_02_Binary_search_for_an_absent_key {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 35));
    }
}
```

4. **Output & Complexity:**

```
-1
```

   Time Complexity: O(log n) time | Auxiliary Space: O(1).

5. **Viva-Voce Questions:** Why is 35 not found? What happens when low becomes greater than high? Can binary search be used directly on an unsorted array?